

物聯網開道器— RFID應用之架構介紹

國立成功大學工程科學系 知識探索實驗室

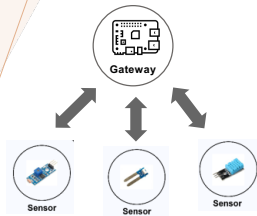
鄧維光、楊皓宇、馬培秀、張馨容

OUTLINE 大綱

RFID閘道器專案介紹及目標

RFID應用之系統架構

結束



物聯網閘道之作用(gateway)

- 物聯網裝置執行任務
像本次RFID應用中RFID Reader為感測裝置，用來讀取標籤。
- 閘道器負責把RFID Reader讀到的標籤訊息，轉換並寫入資料庫，再將狀態顯示在系統端，用於結帳與進出口管理。
- 物聯網中可能會有多個閘道器並管理對應裝置。

客製化目標

本系統旨在建立一套**可擴充的物聯網閘道器架構**，透過 RFID 感測與 MQTT 通訊機制，實現商品／設備在實體空間中的**即時狀態追蹤、行為記錄與異常偵測**。

本次客製化系統解決的問題：

傳統門禁／庫存系統：

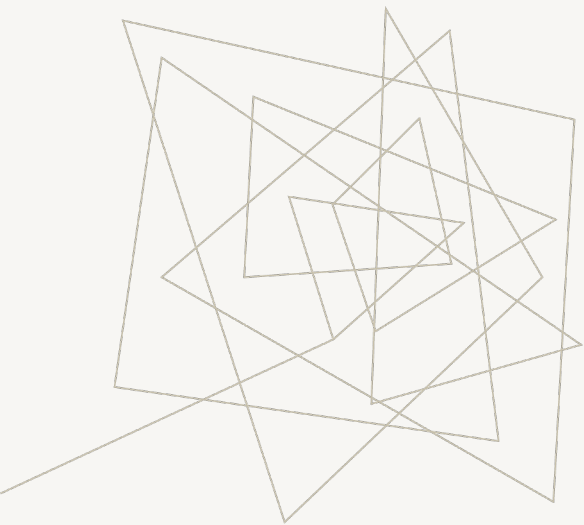
- 邏輯與設備高度耦合
- 難以新增應用情境（如商品、防盜、倉儲）

本系統：

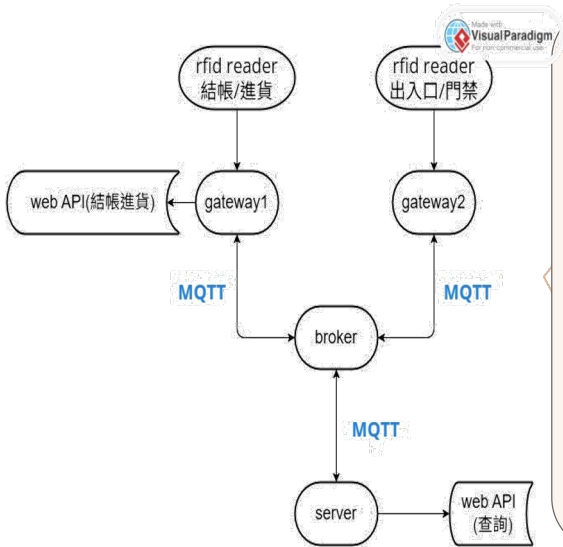
- 將「通訊、任務處理、資料存取」模組化
- 不同應用只需**新增對應模組**即可擴充

系統核心功能

- **RFID 事件接收**：讀取商品或設備的進出入行為
- **狀態管理**：追蹤物品生命週期狀態
 - 0：庫存中（IN_STOCK）
 - 1：已售出（SOLD）
 - 2：未授權離場（UNAUTHORIZED_EXIT）
- **行為紀錄**：
 - 商品操作紀錄（上架、結帳）
 - 進出入／門禁紀錄
- **即時通訊**：透過 MQTT 將事件送往伺服器處理



RFID應用之 系統架構



開放式閘道器模組程式架構

general_gateway/

紅色框起來的為需新增的部分

apps/

inventory/

本次客製化應用

app_executor_inventory.py

db_inventory.py

models /

enum/task_type.py

services/

doorkeeper.py

task_manager.py

fleet_server.sql

藍色字檔案為需修改的程式

fleet_gateway.sql

server_config.ini

gateway_config.ini

(app/templates/Index.html) Web API(操作頁面)

設定檔介紹 (CONFIG/MQTT)

server_config.ini

修改server_id

gateway_id(若有複數台gateway，其id要相異)

broker host

ID不重複即可

依broker實際IP

```
server_config.ini
1 [default]
2 ROLE=Server
3 SERVER_ID=server01
4 GATEWAY_ID=GW01
5 GATEWAY_SN=1
6 BROKER_HOST=192.168.1.50
7 RUN_ENV=flask
8 DELIVER_WAY=Ethernet
9 MYSQL_HOST=iot_mysql
10 MYSQL_PORT=3306
11 MYSQL_USER=root
12 MYSQL_PASSWORD=root
13 DB_NAME=fleet_server
```

gateway_config.ini

```
gateway_config.ini
1 [default]
2 ROLE=Gateway
3 SERVER_ID=server01
4 GATEWAY_ID=GW01
5 GATEWAY_SN=1
6 BROKER_HOST=192.168.1.50
7 RUN_ENV=flask
8 DELIVER_WAY=Ethernet
9 MYSQL_HOST=iot_mysql
10 MYSQL_PORT=3306
11 MYSQL_USER=root
12 MYSQL_PASSWORD=root
13 DB_NAME=fleet_gateway
```

在gateway/server上跑

Doorkeeper.py:負責接收Reader傳來的資料，並準確轉發給Server，確保兩端裝置不需要直接連線也能溝通(不須要修正)

操作頁面介紹

SERVER/GATEWAY1放同個網頁主頁面
但能用的功能不同

Gateway1權限

server權限

商品資訊管理

192.168.0.53:5001

商品資訊管理

進貨 結帳 查詢 查詢RFID出入 查看門禁異常

Gateway1權限

server權限

主頁面

按下後列出status_code為2的商品
(以資料表形式顯示) 以及處理門禁問題的頁面

重新進貨 or 結帳

找到1筆資料

不同的JSON訊息

商品清單

商品ID	商品名稱	價格	最後動作	更新時間	商品狀態
432	123	321.00	add	Tue, 21 Oct 2025 04:29:26 GMT	0

結帳成功

商品清單

商品ID	商品名稱	價格	最後動作	更新時間	商品狀態
6	123	4.00	checkout	Wed, 15 Oct 2025 05:15:03 GMT	1

新增商品 (進貨)

商品ID * 1233

商品名稱 * 333

價格 5523

新增後頁面顯示新增的商品
(以資料表形式顯示)

查詢RFID出入資訊

商品ID (RFID Tag)

查詢後頁面放出Operation Log表
(以資料表形式顯示)

搜尋商品

搜尋關鍵字

輸入商品ID或名稱

搜尋後頁面顯示ID相同或類似的商品
(以資料表形式顯示)

結帳 (修改標籤狀態)

商品ID *

結帳後頁面顯示結帳的商品
(以資料表形式顯示)

WEB API介紹

寫入型 API (Gateway1端頁面使用)

進貨

POST /api/inventory/add

- 參數：tag_id, product_name, price, operator, timestamp
- 說明：產生 ADD_ITEM task_type，送入訊息系統

結帳

POST /api/inventory/checkout

- 參數：tag_id, operator, timestamp
- 說明：產生 CHECKOUT_ITEM task_type，送入訊息系統

查詢型 API (server端頁面使用)

查詢商品

GET /api/inventory/product_item

- 參數：tag_id / keyword
- 說明：查詢商品狀態 (只讀)

查詢操作紀錄

GET /api/inventory/operation_logs

- 說明：查詢進貨 / 結帳紀錄

查詢門禁異常

GET /api/inventory/gate_logs

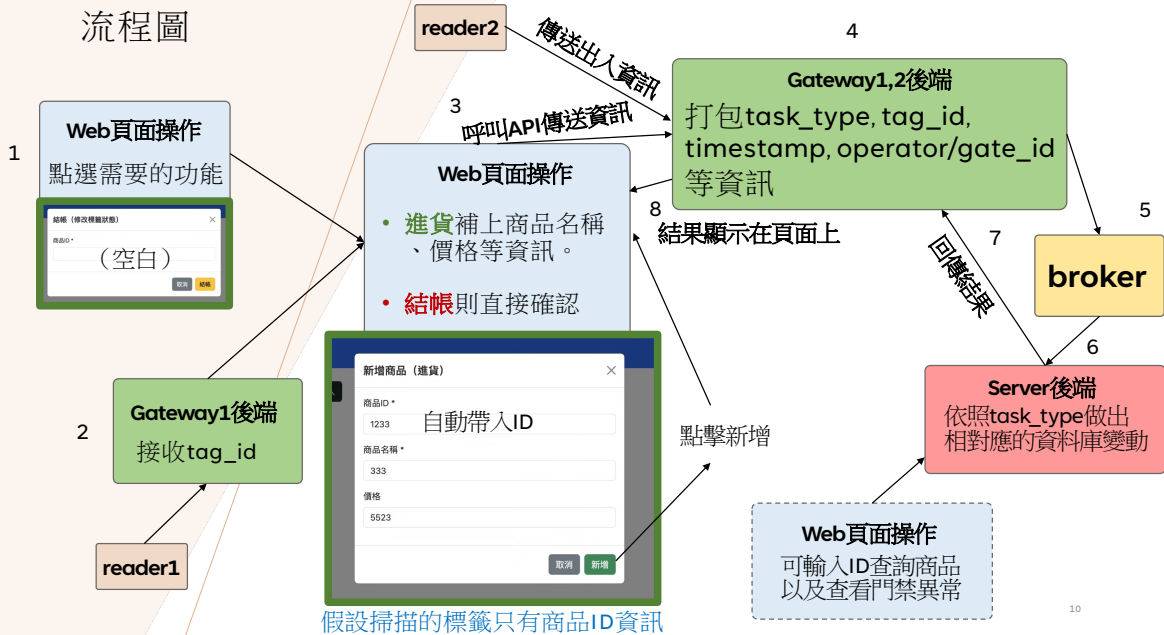
- 說明：查詢未授權離場紀錄

處理門禁異常

POST /api/inventory/process

- 參數：tag_id, status_code
- 說明：處理門禁異常問題

流程圖



TASK_TYPE介紹

系統層 task_type 僅定義 3 種核心任務：

task_type <	功能名稱	對應前端操作	實際行為
ADD_ITEM	進貨	「進貨」按鈕	新增或更新商品為在庫中 (status_code=0)
CHECKOUT_ITEM	結帳	「結帳」按鈕	更新商品為已售出 (status_code=1)
GATE_SCAN	門禁事件	reader 上報	寫入 Gate Log；若未結帳則更新主表為異常 (status_code=2)

網頁上的「查詢 / 查 RFID / 查看異常」
僅為資料查詢行為，不屬於 task_type

狀態碼	意義
0	IN_STOCK (庫存中)
1	SOLD (已結帳)
2	UNAUTHORIZED_EXIT (未授權離場)

TASK_TYPE 任務處理流程 (GATEWAY / SERVER 分工)

流程 1,2：進貨 (ADD_ITEM) / 結帳 (CHECKOUT_ITEM)

[Reader1 掃描標籤]



[Gateway1]

- 取得 tag_id
- 使用者於 Web 補商品資訊
- 組合 task_type



[送出 MQTT Task]



[Server]

- 判斷 task_type
- 呼叫 db_inventory



[資料庫]

回傳流程成功訊息

流程 3：門禁掃描 (GATE_SCAN)

[Reader2 掃描]



[Gateway2]

- 取得 tag_id
- 組合 task_type = GATE_SCAN



[送出 MQTT Task]



[Server]

- 判斷商品當前狀態



[資料庫]

回傳流程成功訊息

TASK_TYPE動作介紹

1.進貨

- Product Item：status_code 變成 IN_STOCK(0)（新商品就 insert、舊商品就 update）
- Operation Log：新增一筆 action=add
- 更新 last_action、update_time

2.結帳

Product Item：status_code 變成 SOLD(1)

- Operation Log：新增一筆 action=checkout
- 更新 last_action、update_time

3.門禁檢查（出口掃描）

- Gate Log：一定要新增一筆 result=pass/unauthorized
- 如果 unauthorized：同步把 Product Item的 status_code 改成 UNAUTHORIZED_EXIT(2)

狀態碼	意義
0	IN_STOCK（庫存中）
1	SOLD（已結帳）
2	UNAUTHORIZED_EXIT（未授權離場）

task_type	功能名稱	對應前端操作	實際行為
ADD_ITEM	進貨	「進貨」按鈕	新增或更新商品為在庫中（status_code=0）
CHECKOUT_ITEM	結帳	「結帳」按鈕	更新商品為已售出（status_code=1）
GATE_SCAN	門禁事件	reader 上報	寫入 Gate Log；若未結帳則更新主表為異常（status_code=2）

資料庫介紹PART1

SQL檔名稱

Server: **fleet_server.sql**

Gateway: **fleet_gateway.sql**

固有資料庫

mqtt_XXXXX

用途：本地任務佇列
/訊息表

Server, gateway都有

資料庫關係

A、**app_product_item** (主表)

—— B、**app_operation_log** (事件表)

—— C、**app_gate_log** (事件表)

商品主表僅負責狀態維護，
所有行為皆以事件形式記錄
於 log 表，有助於系統追蹤
、除錯與後續分析。

A, B, C表新增在server上
方便統整資料

資料庫功能

A、**app_product_item** (商品主表)

用途：每個 tag_id 只會有一筆

只存「現在是什麼狀態」
「最後一次操作是什麼」
「最後更新時間」

👉 判斷商品狀態

B、**app_operation_log** (操作紀錄表)

用途：一個 tag_id 可以有很多筆
每次「進貨 / 結帳」都會新增一筆

👉 判斷商品做過哪些操作

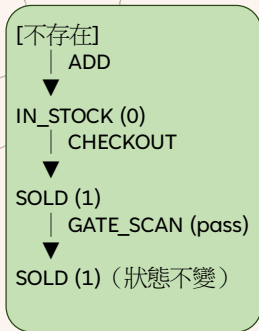
C、**app_gate_log** (門禁紀錄表)

用途：一個 tag_id 可以有很多筆
記錄出口開道的 RFID 掃描事件

👉 用於 判斷未結帳離場 (status_code 2)

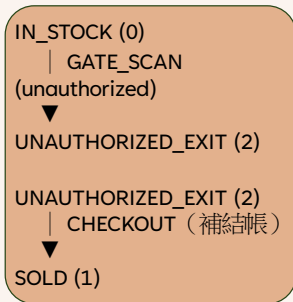
資料庫介紹PART2

商品狀況轉換



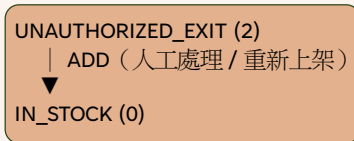
正常進貨流程

如果沒結帳？



狀態碼	意義
0	IN_STOCK (庫存中)
1	SOLD (已結帳)
2	UNAUTHORIZED_EXIT (未授權離場)

如果沒上架/商品不存在？



狀態轉換僅由 `task_type` 觸發，
查詢行為不影響狀態。

資料表A

A、app_product_item (商品主表)

欄位	單位	說明
tag_id	string / varchar	RFID 標籤 ID (主鍵)
product_name	string	商品名稱
price	float	商品價格
status_code	int	商品狀態 (0/1/2)
last_action	string	最後一次操作
update_time	datetime	最後更新時間

狀態碼 意義

0	IN_STOCK (庫存中)
1	SOLD (已結帳)
2	UNAUTHORIZED_EXIT (未授權離場)

資料表B,C

B、 **app_operation_log** (操作紀錄表)

欄位	單位	說明
tag_id	string / varchar	商品 RFID
action	string	add / checkout
operator	string / int	操作者
timestamp	datetime	操作時間

C、 **app_gate_log** (門禁紀錄表)

欄位	單位	說明
tag_id	string / varchar	商品 RFID
gate_id	string / int	出口閘道
result	string	pass / unauthorized
timestamp	datetime	掃描時間

在server上跑

```
- inventory/  
— app_executor_inventory.py  
— db_inventory.py
```

主要程式介紹

app_executor_inventory.py

- 依 gateway 給的 **task_type** 分流
- 呼叫 **db_inventory.py** 的函式並執行

db_inventory.py

- 專門負責 inventory 模組所需的資料庫操作函式，包含商品狀態更新與事件紀錄。

本專題在既有的 IOT 閘道器框架下，inventory 應用模組程式結構完全沿用既有模組設計，包含任務處理層與資料存取層，以支援 RFID 商品管理情境。

程式函式介紹PART1

app_executor_inventory.py

角色：任務處理與業務流程控制

位置：Server 端

Function	Input	Output	說明
handle_task	task	success / error	任務入口
validate_task	task_type, tag_id, timestamp	valid / invalid	基本欄位檢查
dispatch_task	task_type	success / error	依 task_type 分流
handle_add_item	tag_id, operator, timestamp	success / error	進貨（狀態→IN_STOCK）
handle_checkout_item	tag_id, operator, timestamp	success / error	結帳（狀態→SOLD）
handle_gate_scan	tag_id, gate_id, timestamp	success / error	門禁檢查（必要時標記異常）
publish_task_result	result, status_code	success / error	將處理結果回傳至發送端 gateway

Executor 僅負責業務流程判斷與狀態轉換，
實際資料寫入由 db_inventory 處理，
任務結果回傳則由通訊層透過 MQTT 發送。

程式函式介紹PART2

db_inventory.py

角色：資料存取層（資料庫操作）

位置：Server 端

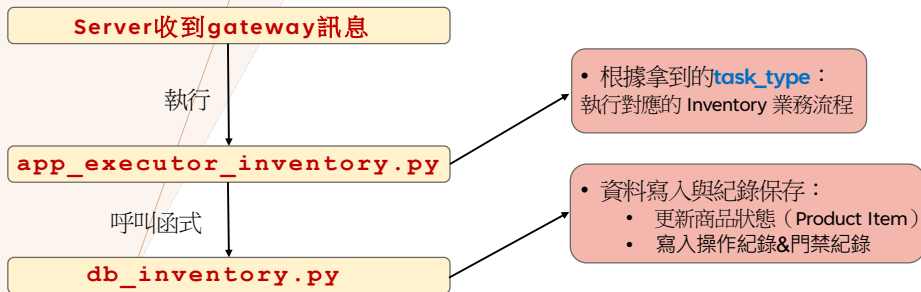
Write Functions（給 app_executor(p17) 用）

Function	Input	Output	說明
get_item	tag_id	item / None	取得商品目前狀態
insert_item	tag_id, price, product_name, status_code, timestamp	success / error	新增商品
update_item_status	tag_id, status_code, last_action, timestamp	success / error	更新商品狀態
insert_operation_log	tag_id, action, operator, timestamp	success / error	寫進貨 / 結帳紀錄
update_gate_log	tag_id, result, gate_id, timestamp	success / error	新增/修改門禁紀錄

Read Functions（給 server Web API 用）

Function	Input	Output	說明
query_items	tag_id	item list	查商品狀態
query_operation_logs	tag_id	log list	查操作紀錄
query_abnormal_items	status_code = 2(UNAUTHORIZED_EXIT)	item list	查門禁異常
handle_abnormal_items	tag_id	success / error	處理門禁異常

程式流程圖



Executor 負責業務流程判斷，
DB 負責資料一致與歷史紀錄。

Abstract geometric lines in a light brown color, forming various polygons and overlapping shapes on the left side of the slide.

THANK YOU